COLLEGE DE PARIS SUPERIEUR

Note : /20

CdPS	Contrôle Continu	Durée : 2h Credit: 2
Année Scolaire : 2024-2025	Épreuve : ARCHITECTURE ET PROGRAMMATION EN ASSEMBLEUR	Classe : B2 - B (IT)

Instructions Générales

- Lisez attentivement chaque question avant de répondre.
- Respectez les consignes spécifiques données pour chaque exercice.
- Les programmes doivent être écrits en syntaxe NASM pour l'architecture x86_64.
- La rigueur syntaxique, la clarté, la lisibilité du code et les commentaires sont pris en compte dans la notation.

Partie I : Questions de compréhension (10 pts)

1. (1 pt) Citez deux différences majeures entre un microprocesseur et un microcontrôleur.
2. (1 pt) Qu'est-ce qu'un registre dans un microcontrôleur AVR ? Donnez un exemple typique.
3. (1 pt) Quels sont les avantages d'utiliser un microcontrôleur AVR dans les systèmes embarqués ?
4. (1 pt) Faire la différence entre un assembleur et un compilateur. ?
5. (1 pt) Quelle est la différence entre la mémoire Flash et la SRAM dans un microcontrôleur AVR ?
6. (1 pt) Expliquez le rôle de l'instruction suivante : `mov rax, 2`
7. (1 pt) Que représente le registre `RIP` dans l'architecture x64 ? Pourquoi est-il crucial dans le déroulement d'un programme ?
8. (1 pt) Quelle est la différence entre une instruction d'accès mémoire directe et indirecte ?
9. (1 pt) En assembleur x64 NASM, que signifie le mot-clé `section .text` ? Quelle autre section courante connaît-on ?
10. (1 pt) Expliquez brièvement le rôle de l'instruction `ret`.

Partie II : Exercices pratiques (10 pts)

Complétez le programme NASM en assembleur x64 selon les instructions données dans les différents exercices. Utilisez le format suivant pour chaque exercice :

```
section .data
    ; Déclaration des variables (si nécessaire)
    ; Par exemple : var1 db 10  ; Déclaration d'une variable de type byte avec la valeur 10

section .text
global main
main:
    ; Instructions à compléter
    ; Par exemple : mov rax, [var1]   ; Charger la valeur de la variable dans un registre
    ; Vous pouvez ajouter des instructions comme l'addition, la soustraction,
    ; ou des déplacements de valeurs.

    ret
```


Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net

Exercice 1 : Soustraction de deux registres (2 pts)

Écrivez un programme NASM en assembleur x64 qui :

- place la valeur 10 dans le registre `RAX`,
- place la valeur 4 dans le registre `RBX`,
- effectue la soustraction `RAX - RBX` et stocke le résultat dans `RCX`.

Exercice 2 : Débogage de programme (2 pts)

Le code suivant contient des erreurs. Corrigez-le pour qu'il fonctionne sous NASM x64 (Windows 64 bits).

```
section .text
global main
main:
    mov rax = 8
    mov rbx, 2
    sub rax rbx
    mov rcx rax
    ret
```

Exercice 3 : Compilation sous Windows (2 pts)

- Décrivez les étapes pour compiler un fichier assembleur NASM (`prog.asm`) sous Windows avec `NASM` et `gcc (MinGW-w64)`.
- Citez deux commandes à utiliser dans le terminal Windows (`cmd` ou `PowerShell`).

Exercice 4 : Utilisation de variable et adressage mémoire (2 pts)

Écrivez un programme NASM x64 sous Windows qui :

- déclare une variable `valeur` contenant l'entier 50,
- charge cette valeur dans le registre `RAX`,
- effectue une multiplication par 2 et stocke le résultat dans `RBX`.

Exercice 5 : Transfert mémoire registre-variable (2 pts)

Complétez le programme suivant pour :

- charger la valeur 25 dans `RAX`,
- stocker cette valeur dans une variable appelée `memo`,
- recharger la valeur de `memo` dans `RBX`.

"Le secret du succès est de faire les choses communes exceptionnellement bien." John D. Rockefeller

Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net

COLLEGE DE PARIS SUPERIEUR

Note : /20

CdPS	Corrigé-type du Contrôle Continu	Durée : 2h Credit: 2
Année Scolaire : 2024-2025	Épreuve : ARCHITECTURE ET PROGRAMMATION EN ASSEMBLEUR	Classe : B2 - B (IT)

Partie I : Questions de compréhension (10 pts)

1. **Citez deux différences majeures entre un microprocesseur et un microcontrôleur.**
   - Le microprocesseur (CPU) est une unité de calcul seule, nécessitant des périphériques externes (RAM, EEPROM, timers, etc.), tandis que le microcontrôleur est une solution tout-en-un embarquant CPU, mémoire (RAM, Flash, EEPROM) et périphériques (USART, I2C, SPI, timers, ADC...).
   - Les microprocesseurs sont souvent utilisés dans des systèmes complexes (PC, serveurs), tandis que les microcontrôleurs s'intègrent dans les systèmes embarqués à ressources limitées.
   - Le microcontrôleur consomme moins d'énergie et est optimisé pour des tâches spécifiques.
2. **Qu'est-ce qu'un registre dans un microcontrôleur AVR ? Donnez un exemple typique.**
   Un registre est une petite mémoire interne très rapide utilisée par le CPU pour exécuter des opérations. Exemple typique : `R16` dans un AVR 8 bits, utilisé pour stocker des valeurs intermédiaires ou des résultats d'opérations arithmétiques.
3. **Quels sont les avantages d'utiliser un microcontrôleur AVR dans les systèmes embarqués ?**
   - Faible coût et faible consommation énergétique.
   - Facilité de programmation (grâce à AVR-GCC et Atmel Studio).
   - Large écosystème avec des bibliothèques, outils de développement, et documentation.
   - Intégration de nombreux périphériques internes (USART, ADC, PWM. . .).
4. **Faire la différence entre un assembleur et un compilateur.**
   - Un **assembleur** traduit le code assembleur (langage bas niveau proche du binaire) en langage machine spécifique à une architecture (ex. NASM pour x86/x64).
   - Un **compilateur** traduit un langage de haut niveau (C, Python...) en langage machine, souvent via une étape intermédiaire en assembleur.
5. **Quelle est la différence entre la mémoire Flash et la SRAM dans un microcontrôleur AVR ?**
   - **Flash** : mémoire non-volatile utilisée pour stocker le programme (ROM modifiable).
   - **SRAM** : mémoire volatile utilisée pour stocker les données et variables pendant l'exécution du programme.
6. **Expliquez le rôle de l'instruction suivante :** `mov rax, 2`
   Elle place la valeur immédiate `2` dans le registre `RAX` (registre 64 bits du processeur).
7. **Que représente le registre `RIP` dans l'architecture x64 ? Pourquoi est-il crucial ?**
   `RIP` (Register Instruction Pointer) contient l'adresse de la prochaine instruction à exécuter. Il permet de suivre le flot d'exécution du programme.
8. **Quelle est la différence entre une instruction d'accès mémoire directe et indirecte ?**
   - Accès **direct** : l'adresse de la donnée est spécifiée explicitement. `mov rax, [0x4000]`
   - Accès **indirect** : l'adresse est contenue dans un registre. `mov rax, [rbx]`
9. **En assembleur x64 NASM, que signifie le mot-clé `section .text` ? Quelle autre section courante connaît-on ?**
   `section .text` contient le code exécutable. Autres sections :
   - `.data` : variables initialisées.
   - `.bss` : variables non initialisées.
10. **Expliquez brièvement le rôle de l'instruction `ret`.**
    Elle provoque le retour d'une fonction en restaurant l'adresse de retour stockée sur la pile (stack).

Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net

Partie II : Exercices pratiques (10 pts)

Exercice 1 : Soustraction de deux registres (2 pts)

```
section .text
global main
main:
    mov     rax, 10         ; Charger 10 dans RAX
    mov     rbx, 4          ; Charger 4 dans RBX
    sub     rax, rbx        ; RAX ← RAX - RBX = 6
    mov     rcx, rax        ; RCX ← résultat (6)
    ret
```

Exercice 2 : Débogage de programme (2 pts)

Code corrigé :

```
section .text
global main
main:
    mov     rax, 8          ; Utiliser une virgule, pas '='
    mov     rbx, 2
    sub     rax, rbx        ; Corriger la syntaxe de sub
    mov     rcx, rax
    ret
```

Exercice 3 : Compilation sous Windows (2 pts)

Étapes de compilation avec NASM et gcc (MinGW-w64) :

1. Assembler le fichier source NASM en fichier objet 64 bits :

   `nasm -f win64 prog.asm -o prog.obj`

2. Lier avec GCC pour produire un exécutable :

   `gcc prog.obj -o prog.exe`

Exercice 4 : Utilisation de variable et adressage mémoire (2 pts)

```
section .data
    valeur dq 50          ; Déclaration d'un entier 64 bits

section .text
global main
main:
    mov     rax, [valeur] ; Charger 50 dans RAX
    shl     rax, 1        ; Multiplication par 2 (équiv. à ×2)
    mov     rbx, rax      ; Stocker le résultat dans RBX
    ret
```


Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net

Exercice 5 : Transfert mémoire registre-variable (2 pts)

```
section .data
    memo dq 0              ; Réservation d'un espace mémoire pour 64 bits

section .text
global main
main:
    mov     rax, 25        ; Charger 25 dans RAX
    mov     [memo], rax    ; Stocker la valeur de RAX dans memo
    mov     rbx, [memo]    ; Charger la valeur de memo dans RBX
    ret
```


Dr AFENYIVEH Dzo M. Serge
d.afenyiveh@univkara.net